

Lab09 - Elliptic Curve Diffie-Hellman Key Exchanges

You will be using OpenSSL to perform Elliptic Curve Diffie-Hellman Ephemeral (ECDHE) key exchanges. This will involve generating elliptic curve key pairs, exchanging public keys, deriving shared secrets, and using the derived secrets to encrypt and decrypt data. The goal is to understand forward secrecy and how ephemeral keys work. You will interact with an OpenSSL server configured to prioritize ECDHE key exchanges and observe the TLS handshake in verbose mode to help understand the key exchange details.

Instructions for submission are listed at the end of the document. All exercises will be performed on your **Ubuntu 24.04 desktop IN ISELab**. **Note:** you'll need to perform all of the steps in order.

*all instances of <netid> should be replaced by **YOUR** netid:

vi <netid>_msg.txt ⇒ vi jrursch_msg.txt

Part 01- Generate ec key pair and certificate

1. First, you will need to create a working directory to hold the documents generated during this homework. Create a new directory and change-directory into it:
 - a. `mkdir ~/homework/lab09`
 - b. `cd lab09`
2. Next, generate a public/private key pair on your machine using `openssl`. Python3 could have been used, but `openssl` is a real-world tool that you should gain familiarity with. To generate the private key, execute the following command:
 - a. `openssl ecparam -genkey -name secp384r1 -out <netid>_private_key.pem`
 - b. This will generate a single file containing your private key in PEM format. You can `cat` the file to read the private key in base64.
3. Extract your public key from the private key:
 - a. `openssl ec -pubout -in <netid>_private_key.pem -out <netid>_public_key.pem`
4. View all of your private key's data with:
 - a. `openssl ec -text -in <netid>_private_key.pem | less`
 - b. This will show the private key in hex, the public key and two names for the curve. **Submit a screenshot** of the output of this command.
 - c. When viewing the details of your private key with OpenSSL, you can see both an 'ASN1 OID' and a 'NIST Curve' name. The ASN1 OID appears as 'secp384r1,'

while the NIST Curve is listed as 'P-384.' **What is the difference between these two identifiers?** You may want to have Google/AI help you with this answer, but don't copy and paste from Google (or any AI help). *Put it into your OWN WORDS, please.*

5. Create a certificate signing request and submit it to the ECDHE certificate authority (`ecdhecerts.homework.3310.com`)
 - a. Make sure you are in your lab09 folder: `cd ~/homework/lab09`
 - b. `openssl req -new -key <netid>_private_key.pem -out <netid>_certificate.csr -days 3650`
 - c. You should be prompted to answer a series of questions. Ensure that you answer these as follows:
 - i. Country Name: US
 - ii. State or Province Name: Iowa
 - iii. Locality Name: Ames
 - iv. Organization Name: 3310.com
 - v. Organizational Unit Name: ecdhe.homework
 - vi. Common Name: <netid>.ecdhe.homework.3310.com
 - vii. Email Address: <netid>@ecdhe.homework.3310.com
 - d. Ensure that you **do not** enter a challenge password or optional company name. Just press "Enter" when presented with these fields. If you mess up, you can always cancel with "Ctrl+C"
6. Upload your CSR to be signed, similar to working with RSA last week. Remember, ftp is OLD. I am using it for simplicity. In the real world/production environment this would be done over https or using an API.
 - a. Start the interactive FTP program: `ftp`
 - b. Open a connection: `open ecdhecerts.homework.3310.com`
 - c. Log in as: `anonymous` (Password is also `anonymous`)
 - d. Change directory to: `cd upload_cert_req`
 - e. Upload your CSR: `put <netid>_certificate.csr`
 - f. Note: You won't be able to do this twice. If you make a mistake and have to do it over, reach out to the TA. They must remove you from the certificate database (`index.txt`), as well as remove your `csr` and `pem` from the corresponding `ftp` directories.
7. Retrieve your signed certificate
 - a. While still logged into FTP, change directory to: `download_cert` via the command:
 - i. `cd ../download_cert`
 - b. Retrieve your certificate with: `get <netid>_certificate.pem`
 - i. It may take a minute or so to generate your certificate in the directory.
 - c. Also get the `ca_cert.pem` for Part 02
 - i. `cd ../download_ca_cert`
 - ii. `get ca_cert.pem`
 - d. Close connection

- i. Close
 - ii. quit
- 8. View your certificate
 - a. Run the following command to view the contents of your newly signed certificate:
 - b. `openssl x509 -in <netid>_certificate.pem -text -noout`
`|less`

Part 02 - Demonstrate Shared Secret in ECDHE and the TLS Handshake

The server `openssl.homework.3310.com` has been configured as an OpenSSL server that supports ECDHE key exchanges and kicks off a basic web response. It has its own server certificate signed by our CA and a private key. You will use this server to establish a TLS connection in which a shared session key is created. You can't see the actual shared key (that whole security thing), but you can see the components used in ECDHE that create the session key.....think the water demonstration I did in lecture a few weeks ago.

1. Establish the connection to the OpenSSL server. This will give debug information across the terminal and then sit in waiting for additional commands because we are running `openssl` in interactive mode. You can just press `Ctrl+ C` to end the connection.
 - a. `openssl s_client -connect openssl.homework.3310.com:4433`
`-tls1_3 -cipher ECDHE-ECDSA-AES256-GCM-SHA384 -curves`
`P-384 -cert <netid>_certificate.pem -key`
`<netid>_private_key.pem -CAfile ca_cert.pem -sess_out`
`<netid>_session_ticket.bin -showcerts -state -debug -trace`
`-verify_return_error -ignore_unexpected_eof >`
`<netid>_tls_handshake_details.txt`
2. The debug process shows you the multiple steps in the handshake. You will examine the trace details in the text file you captured but for your information I will explain the debug messages
 - a. *SSL initialization*: This indicates the beginning of the SSL/TLS connection process. OpenSSL is preparing to establish a secure connection.
 - b. */TLS write client hello*: The client is sending the ClientHello message to the server. This message includes the list of supported protocols, cipher suites, elliptic curves, and other extensions. It initiates the negotiation process.

- c. */TLS read server hello*: The server has responded with the ServerHello message. This message confirms the server's selected protocol version, cipher suite, and key exchange method (such as ECDHE with the chosen elliptic curve).
 - d. *.3 read encrypted extensions*: In TLS 1.3, after the ServerHello, the server sends a set of encrypted extensions. These extensions include additional parameters agreed upon during the handshake, such as supported protocols and features.
 - e. */TLS read server certificate request*: The server has requested the client's certificate for mutual authentication. The client is expected to provide its certificate in response, if mutual authentication is enabled.
 - f. *depth=1 ... verify return:1* This indicates that OpenSSL is validating the server's certificate chain. Here, it is checking the certificate of the intermediate CA (or a higher-level CA) and confirms its validity (verify return:1).
 - g. *depth=0 ... verify return:1* This shows the validation of the server's certificate itself. Successful verification (verify return:1) confirms that the server's certificate is valid and issued by a trusted CA.
 - h. */TLS read server certificate*: The server's certificate is being read by the client. This certificate should contain the server's public key for authentication purposes.
 - i. *.3 read server certificate verify*: In this step, the server is proving to the client that it possesses the private key corresponding to its certificate. This step is critical for preventing man-in-the-middle attacks.
 - j. */TLS read finished*: The server has completed its part of the handshake. The server's Finished message includes a cryptographic hash of all handshake messages up to this point, ensuring integrity.
 - k. */TLS write change cipher spec*: The client is signaling that it is switching to the newly negotiated cryptographic parameters for secure communication. This step is essential for transitioning to an encrypted state.
 - l. */TLS write client certificate*: The client is sending its certificate to the server. This happens only if mutual authentication is enabled and required by the server.
 - m. */TLS write certificate verify*: The client is proving to the server that it possesses the private key corresponding to the certificate it just sent.
 - n. */TLS write finished*: The client is sending its Finished message, which includes a cryptographic hash of all handshake messages up to this point, ensuring the integrity of the client's part of the handshake.
 - o. *negotiation finished successfully*: The handshake has been successfully completed, and both the client and server have verified each other. They can now securely communicate using the negotiated encryption parameters.
 - p. */TLS read server session ticket*: The server is sending a session ticket to the client. This ticket can be used to resume the session in the future without performing a full handshake, improving efficiency.
3. Open a new terminal. Then open the `<netid>_tls_handshake_details.txt` file
- a. You see the line `CONNECTED(00000003)`.
 - i. **What does this line indicate about the steps that occurred before establishing the SSL/TLS handshake?** Hint: Think back to 2300 and

about how connections are established between a client and the server at the transport layer before any secure communication can happen.

- b. Look at the `ClientHello` header
 - i. You will see TLS 1.0. It is for backwards compatibility. It ensures older versions of TLS can process the initial part of the handshake. It does not mean that the client is using TLS 1.0
 - ii. Then you will see `client_version=0x303` (TLS 1.2) even when the client is trying to only negotiate TLS 1.3. It is a requirement for backward compatibility so that the servers who don't recognize TLS 1.3 won't misinterpret the `ClientHello`.
 - iii. Finally, you will see the `extension_type=supported_versions`. This contains the ACTUAL versions the client supports. Based upon the command I asked you to run, only TLS 1.3 should be included there.
- c. While still in the `ClientHello` section, find the `extension_type=key_share`. Below that, you will see the `NamedGroup` and `key_exchange` values.

- i. **Take a screenshot of the key_share information**

```
extension_type=key_share(51), length=103
  NamedGroup: secp384r1 (P-384) (24)
  key_exchange: (len=97): 048F587646489E2F2181A1F2FF3C1C7B48BE76012EA0260365D71B7B56BB0F42204DE6FF
D75D225FC43C31C440C1F543F396024DD2B4502FDB06479621B03978AF2CCF2491FAFEE356876DBDB0BAD0AB568559466F979C2951351
4CD97336258B6
```

- ii. **What information do you think these fields are providing to the server? How might the server use this information during the TLS 1.3 handshake to establish a secure connection? Does this contribute to perfect forward secrecy? Why or why not?**
 - 1. Hint: Think about how the client and server securely communicate without sending the session key over the network. The client's key in the `key_share` extension needs to work together with something the server generates to create a shared value that both sides can use for encryption.

- d. Search for the `ServerHello` section.
 - i. **Take a screenshot of the supported version extension. What version(s) of TLS does the server actually support? Base your reasoning on the information provided above.**
 - ii. **Take a screenshot of the key_share information.**

```
extension_type=key_share(51), length=101
  NamedGroup: secp384r1 (P-384) (24)
  key_exchange: (len=97): 043B7BB8818985B3AF800A0B8B95C90198CFA4ECC7852FF6CE9D8ADA31C100CC202AA89
7D2983E2DA85D1E665B54FA5027D12641499742ED9F34D09816D644D96BE43B3528CFE0CA8758BA7EF45037536DA21EA4C12FAC4BB772
6FFE05110F9BA
```

- e. Search for the `Certificate chain`. This is for the server. **Take a screenshot.**
 - i. Look at the subject (s) and the issuer (i) for each of the two certificates. **Which number (0 or 1) is the certificate for the Certificate Authority (CA)? Why?**
 - ii. **What happens if the server's certificate validation fails at any depth?**

- f. Search for your netid.
 - i. You will see your certificate being provided for verification.
- g. Search for `SSL handshake`. Below it the cipher being used for the session key is listed. **Expand the acronym for each part and explain the purpose of each.**

```

---
SSL handshake has read 2174 bytes and written 2010 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 384 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---

```

- h. Search for `NewSessionTicket`
 - i. A new session ticket is used to allow the client and server to resume a previous secure session without performing a full handshake. This improves performance by reducing the number of round trips to establish a connection. Some of the information stored in these tickets included the session key. The client stores the tickets locally. When the client wants to resume a session, it can present the ticket to the server and if the server accepts the ticket, both parties skip the full handshake and use a shorter handshake called the PSK (Pre-Shared Key) handshake.
 - ii. When you created your openssl connection, one of the flags was to store your session key. Resume your session using that stored ticket.
 - iii.


```

openssl s_client -connect
openssl.homework.3310.com:4433 -cert
<netid>_certificate.pem -key <netid>_private_key.pem
-CAfile ca_cert.pem -sess_in
<netid>_session_ticket.bin -showcerts -state -debug
-trace

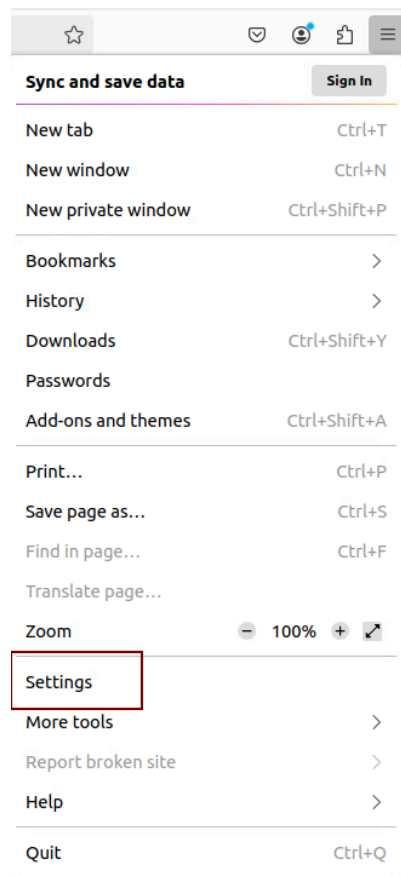
```
 - iv. TLS 1.3 ensures that every session, whether full or resumed, gets new key material. **What does “getting new key material” mean?** Hint: It relates to perfect forward secrecy.
4. You will now view the simple website. You first have to convert your personal certificate to the pkcs12 format where the private key and the cert are stored together. This should sound like a risky idea to you. **Why would creating pkcs12 files be a bad idea generally speaking?**
 - a.


```

openssl pkcs12 -export -in <netid>_certificate.pem -inkey
<netid>_private_key.pem -out
<netid>_certificate_pkcs12.p12

```

- b. Press “Enter” when prompted for a password.
- 5. Now you have to import the `ca_cert.pem` and the `<netid>_certificate_pkcs12.p12` into Firefox.
 - a. If Firefox will not open correctly. Remove it and reinstall it. Then reboot. You should NOT have this problem and can continue to b.
 - i. `sudo apt purge firefox`
 - ii. `sudo snap remove firefox`
 - iii. `sudo apt update`
 - iv. `sudo apt install firefox`
 - v. `reboot`
 - b. Hamburger menu > Settings



- c. Search for certificates > Manage Certificates

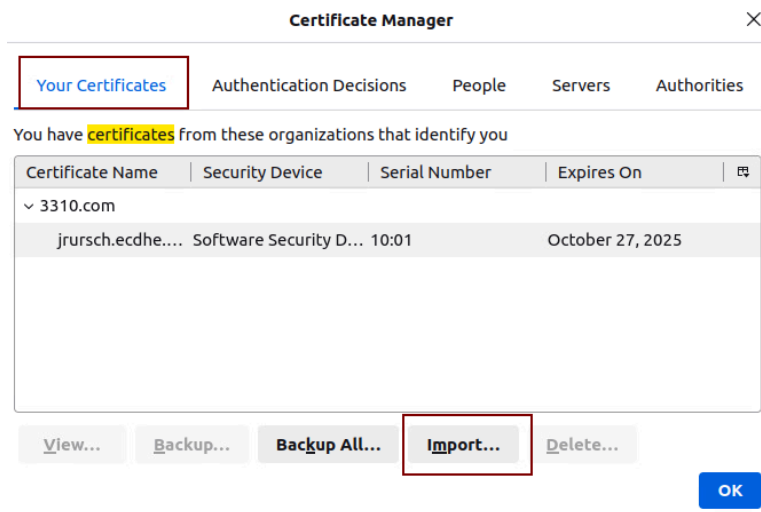
Search Results

Certificates

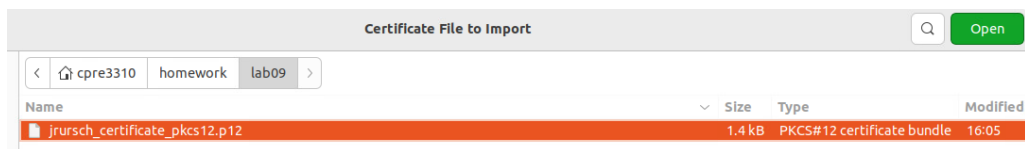
Configure the **certificates** that Firefox uses to verify secure connections. [Learn more](#)

Manage certificates	>
Manage security devices	>

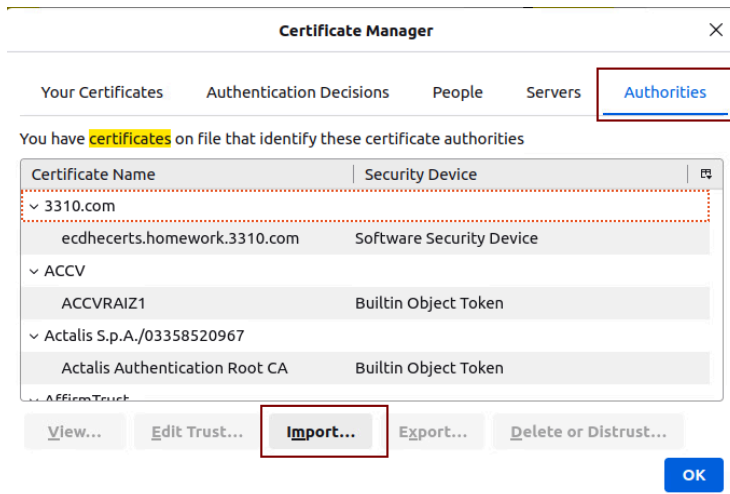
d. Your certificates tab > Import



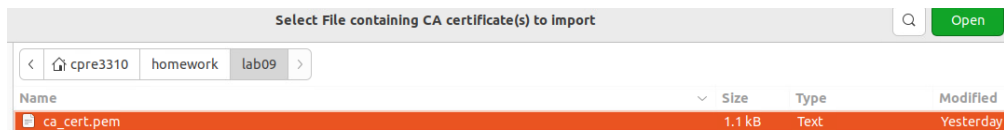
e. Import <netid>certificate_pkcs12.p12



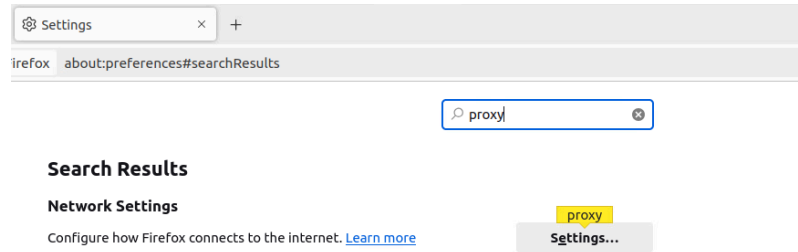
f. Authorities > Import



g. Import ca_cert.pem



h. Search for proxy > Settings



- i. Set no proxy for `openssl.homework.3310.com`



6. Close Firefox because you need to set an environment variable to capture all the key data. You are going to set up key logging to capture the pre-master secret negotiated during the handshake or the session keys material by exporting the log. The can later be opened later with Wireshark to read an encrypted packet capture. This is not a good security practice, but is useful in the lab.
 - a. `export SSLKEYLOGFILE=$HOME/homework/lab09/sslkeys.log`
7. Now, you have to start Firefox from the command line to get the key logging to work
 - a. `firefox &`
8. Use Firefox with the certificates you just added to access the website `https://openssl.homework.3310.com:4433` **Take a screenshot of the ciphers supported in `s_server` binary.**
 - a. **Find and record in your lab report at least one that should no longer be used in the real world. Support your reason why it shouldn't be used.**
9. Close Firefox and open `sslkeys.log`. Notice you don't have the exact session keys, but you have all the key material for Wireshark to decrypt your traffic (if I had a more completely built out website).

Lab 09 Template

Some questions require multiple parts to be answered, be sure to discuss them in full.

Part 01:

1. **Screenshot of the output from** `openssl ec -text -in <netid>_private_key.pem |less`
(5 points)

2. **What is the difference between the two curve identifiers?**
(5 points)
3. **What does the CONNECTED(00000003) line indicate about the steps that occurred before establishing the SSL/TLS handshake?**
(5 points)
4. **Take a screenshot of the client key_share information**
(5 points)
5. **What information do you think the client key_share fields are providing to the server? How might the server use this information during the TLS 1.3 handshake to establish a secure connection? Does this contribute to perfect forward secrecy? Why or why not?**
(10 points total, 2.5 points each question)
6. **Take a screenshot of the supported version extension for the server. What version(s) of TLS does the server actually support?**
(10 points total, 5 points screenshot, 5 points question)
7. **Take a screenshot of the key_share information.**
(5 points)
8. **Take a screenshot of the certificate chain**
(5 points)
9. **Look at the subject (s) and the issuer (i) for each of the two certificates. Which number is the certificate for the Certificate Authority (CA)? Why?**
(10 points total, 5 points for each question)
10. **What happens if the server's certificate validation fails at any depth?**
(5 points)
11. **Expand the acronym for each piece of the cipher suite and explain the purpose of each.**
(10 points total, 2 points each piece)
12. **What does "getting new key material" mean?**
(5 points)
13. **Why would creating pkcs12 files be a bad idea generally speaking?**
(5 points)

14. Take a screenshot of the ciphers supported in s_server binary.

(5 points)

15. Find and record in your lab report at least one that should no longer be used in the real world. Support your reason why it shouldn't be used.

(10 points total, 5 points each answer)